

Talk 3: Fault tolerant resource calcs and optimisations



Tutorial Roadmap

Room 717B

Tuesday September 15, 2026

10.00am

10.45am

12.00pm

1.00pm

1.45pm

2.30pm

1 Intro to
quantum
chemistry

2 From
chemistry to
quantum
computers

Lunch

3 Circuit
optimisations
and resource
estimates

4 Automating
resource
estimation on
software

Overview

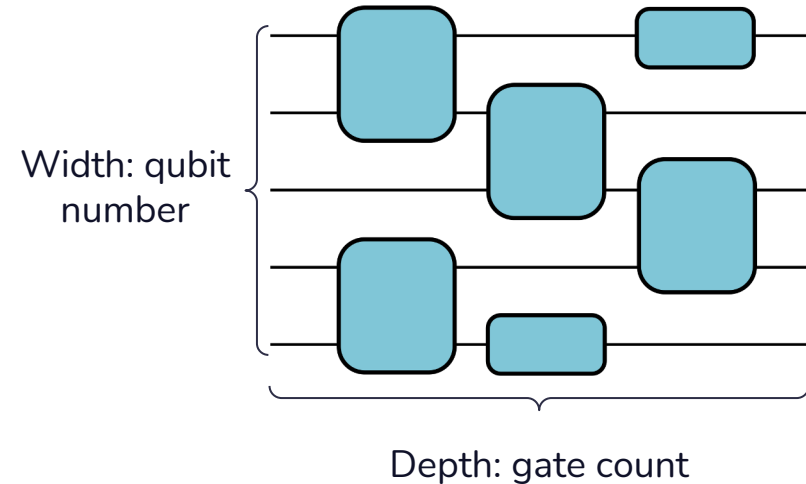
Before lunch:

- Introduction to electronic structure calculations
- Principles of quantum computing for electronic structure

Now:

- Introduction to resource estimation
- Numerical examples building up to a QPE circuit
- Accounting for shots and logical-to-physical conversion

Resource estimation



Algorithm cost $\sim$ width $\times$ depth

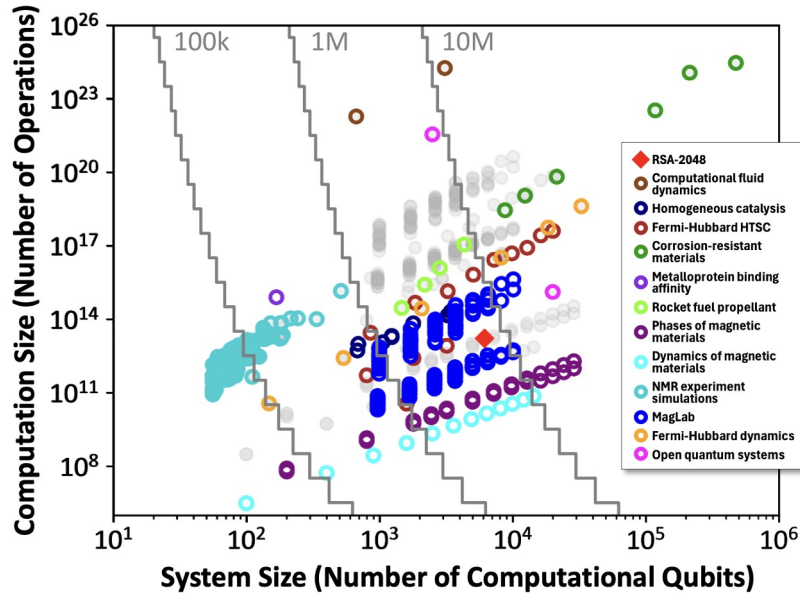
- Quantum gates can be anything, but are broken down into a chosen finite gate set.
- Qubit number counts data and ancilla qubits. Ancilla qubits are often assumed to be clean.
- Width and depth account for logical qubits and gates.

logical $\rightarrow$ physical resource conversion is a crucial part of resource estimation

... also particularly challenging: **Highly dependent on hardware and error correction!**

Resource diagrams

A resource diagram shows qubit and operation numbers for different algorithms:



Source: DARPA / QBI

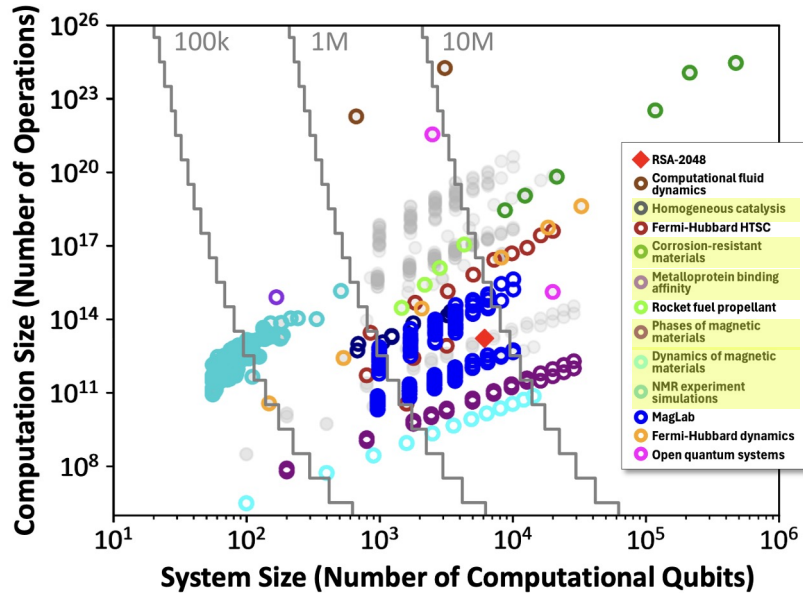
Use cases:

- Assess runtime of algorithms.
- Compare algorithms.
- Determine size of quantum computer necessary to run algorithms.
- Optimise for hardware / runtime constraints.

What's a useful problem that I will be able to solve with a quantum computer made in the year Y?

Resource diagrams

A resource diagram shows qubit and operation numbers for different algorithms:



Source: DARPA / QBI

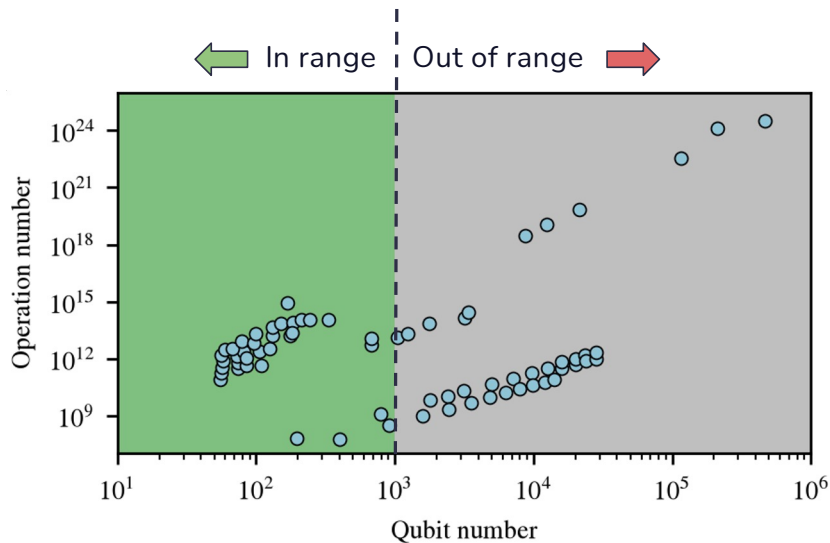
Use cases:

- Assess runtime of algorithms.
- Compare algorithms.
- Determine size of quantum computer necessary to run algorithms.
- Optimise for hardware / runtime constraints.

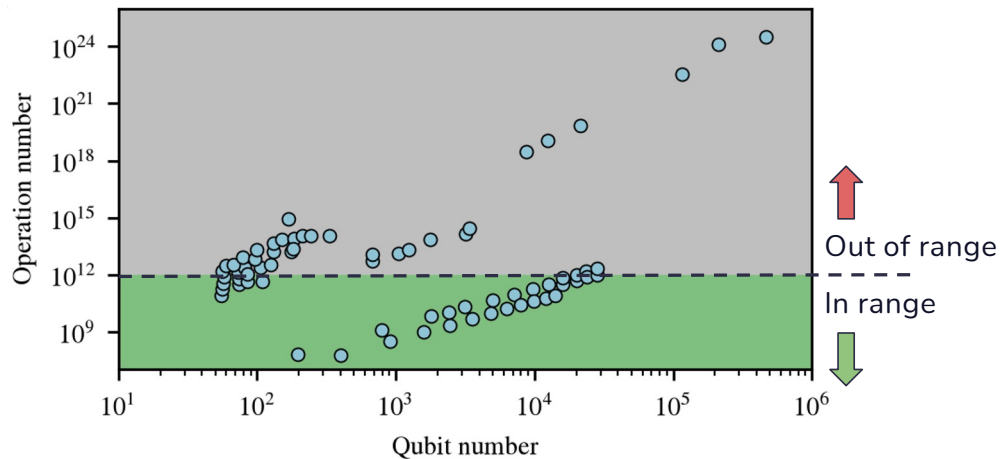
What's a useful problem that I will be able to solve with a quantum computer made in the year Y?

Resource counts to compare algorithms and applications

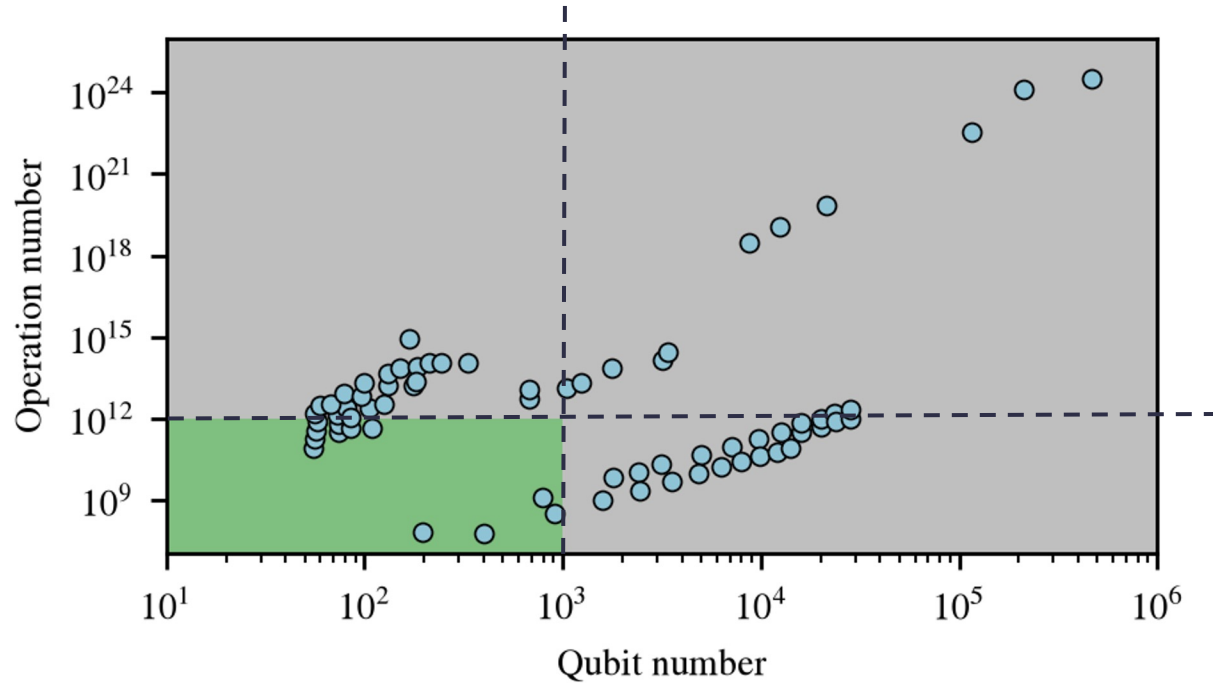
How big does my quantum computer need to be?



What fidelity (and how long) will my quantum computer need to solve problem X?

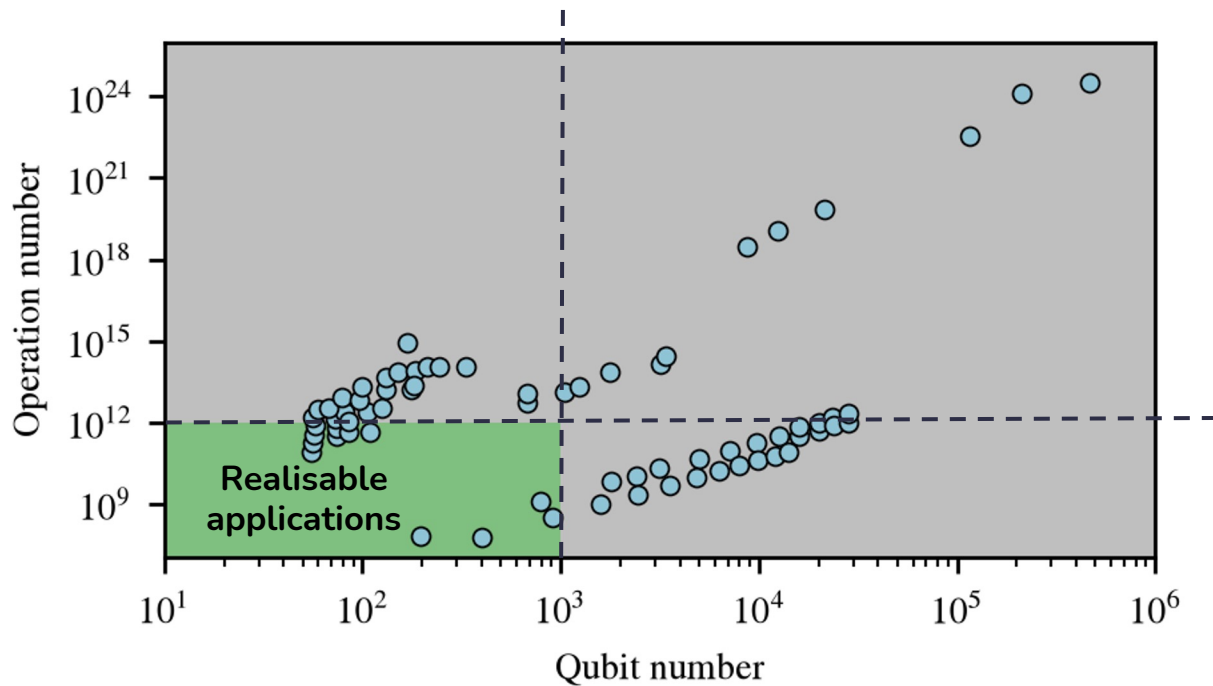


Resource counts for roadmaps



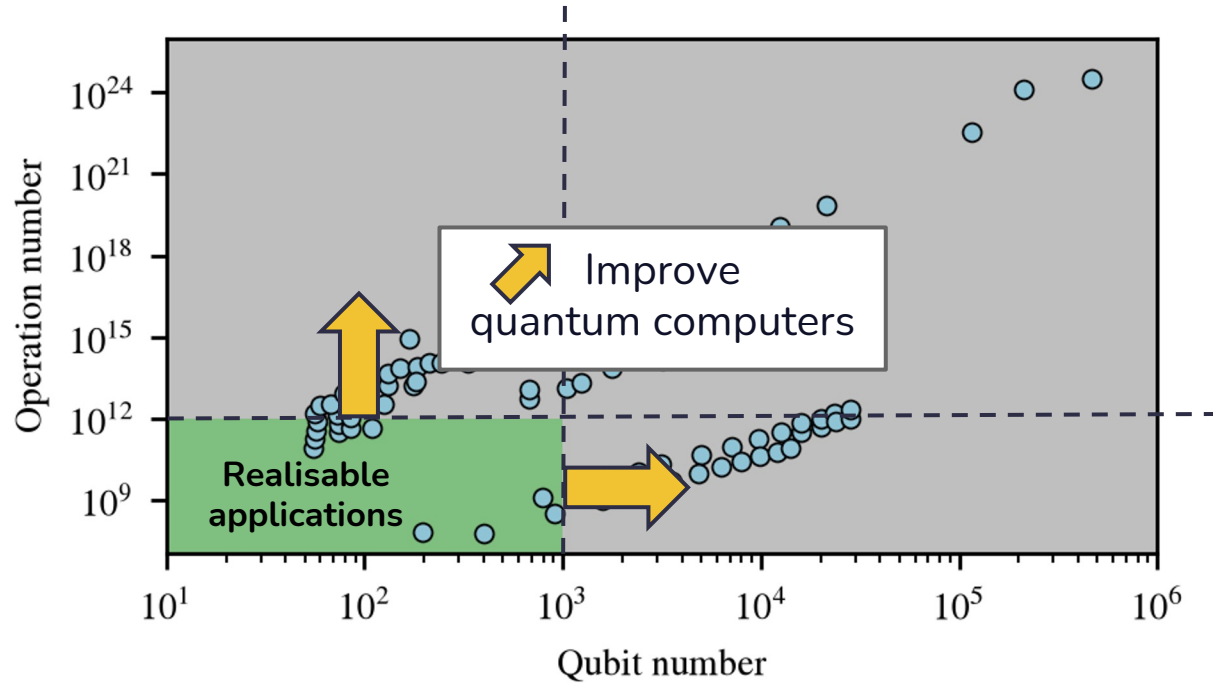
Resource estimation allows us to **today** evaluate the utility of **tomorrow's** quantum computer

Resource counts for roadmaps



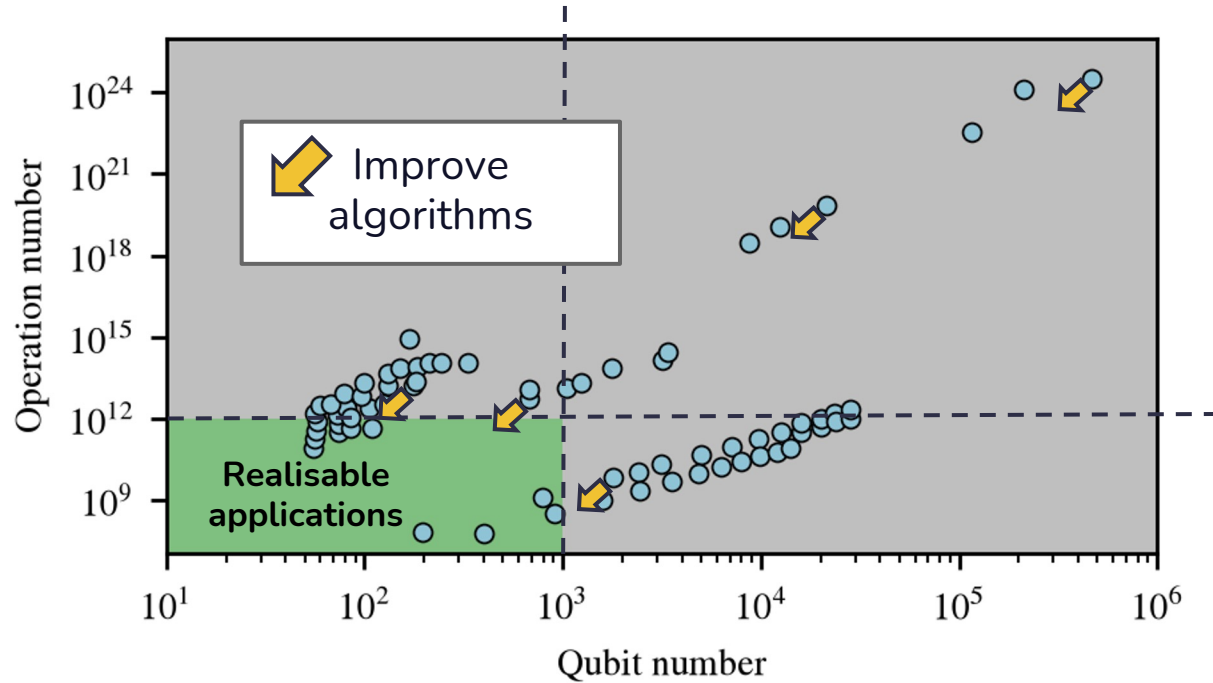
Resource estimation allows us to **today** evaluate the utility of **tomorrow's** quantum computer

Resource counts for roadmaps



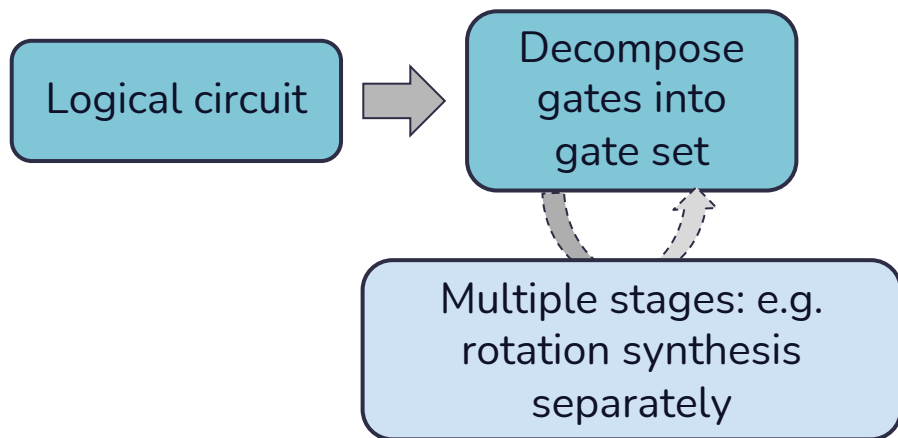
Resource estimation allows us to **today** evaluate the utility of **tomorrow's** quantum computer

Resource counts for roadmaps

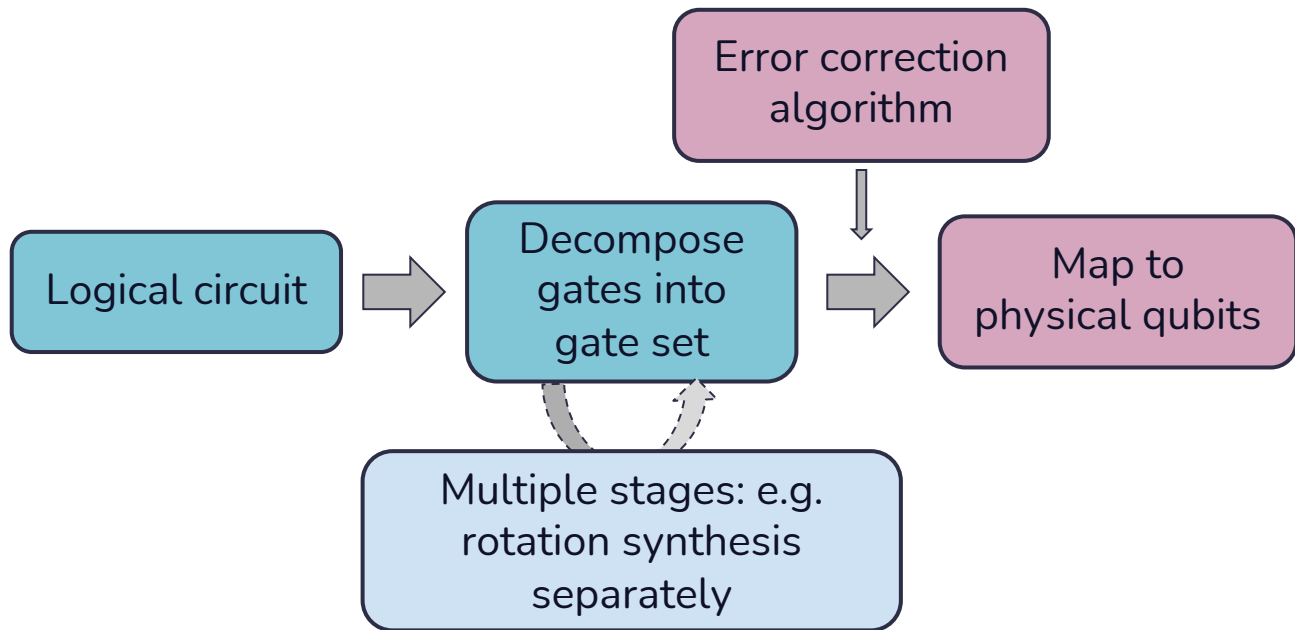


Resource estimation allows us to **today** evaluate the utility of **tomorrow's** quantum computer

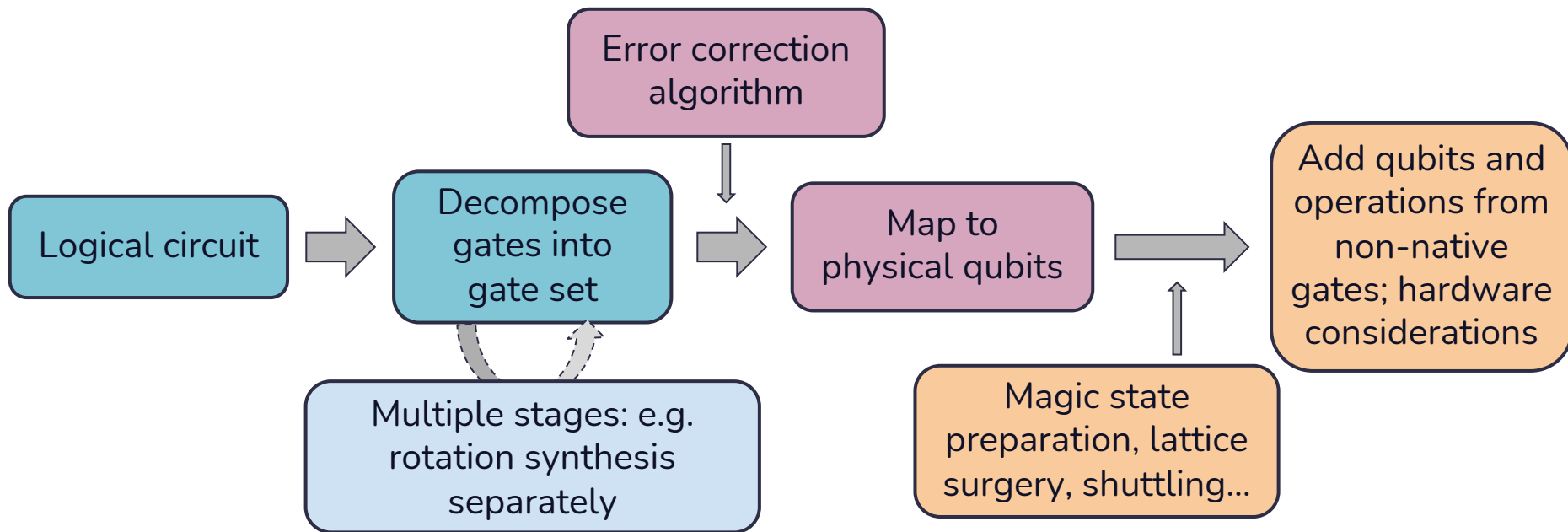
Resource estimation workflow



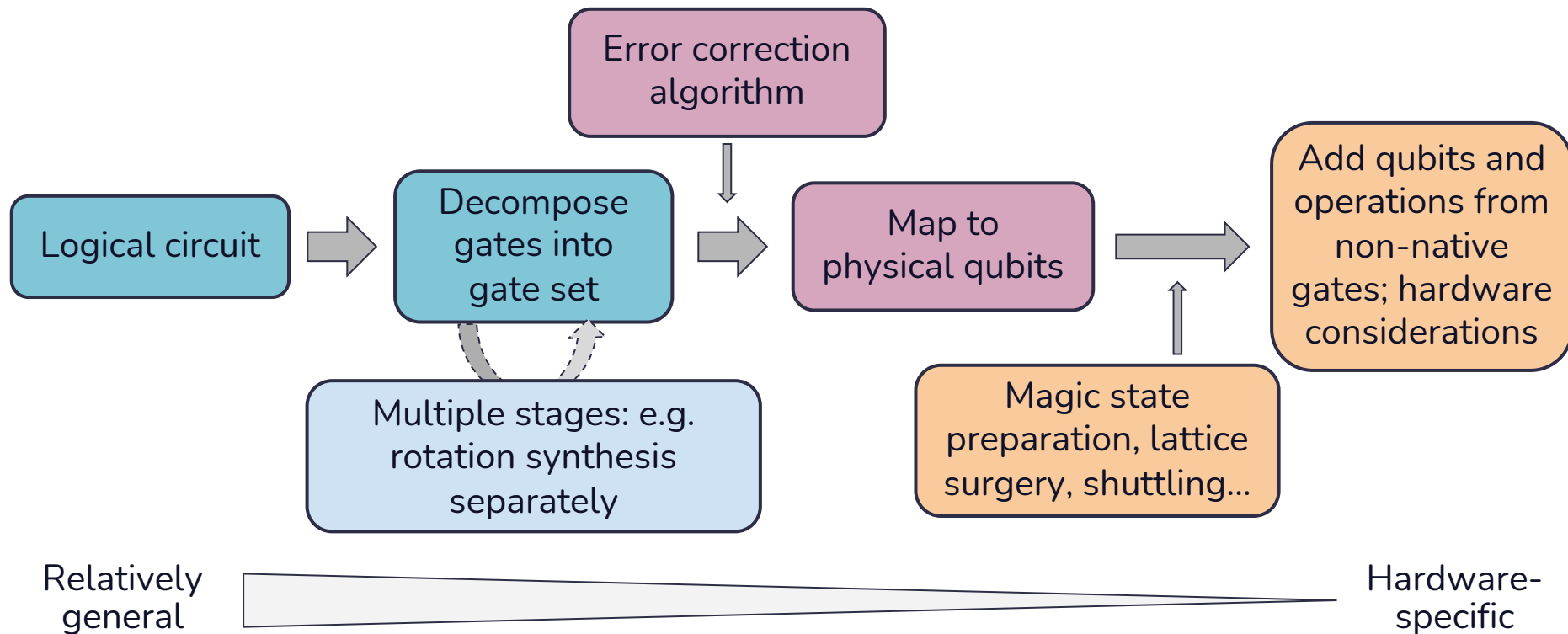
Resource estimation workflow



Resource estimation workflow

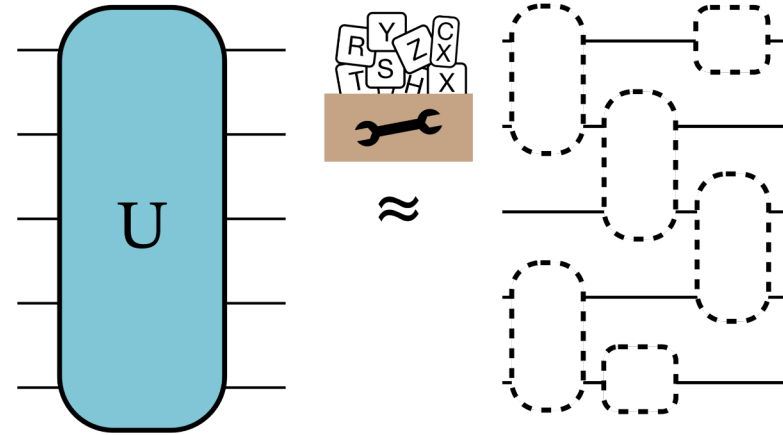


Resource estimation workflow



Decomposition into basic gate set

- Fault-tolerant computers can apply a finite set of gates to high precision.
 - Often: Pauli gates, H, S, T gate.
- All gates are broken down into this gate set.
- Depending on the error correction approach, certain gates are easy to implement while others are hard.
- Resource estimation often focuses on the ‘hard’ gates.



... also allows for **direct comparison** of different algorithms.

Gate set for the surface code

Clifford gates

- Pauli X, Y, Z gates
- Hadamard gate H
- Phase gate S
- CNOT ← Two-qubit gate

Limiting gates

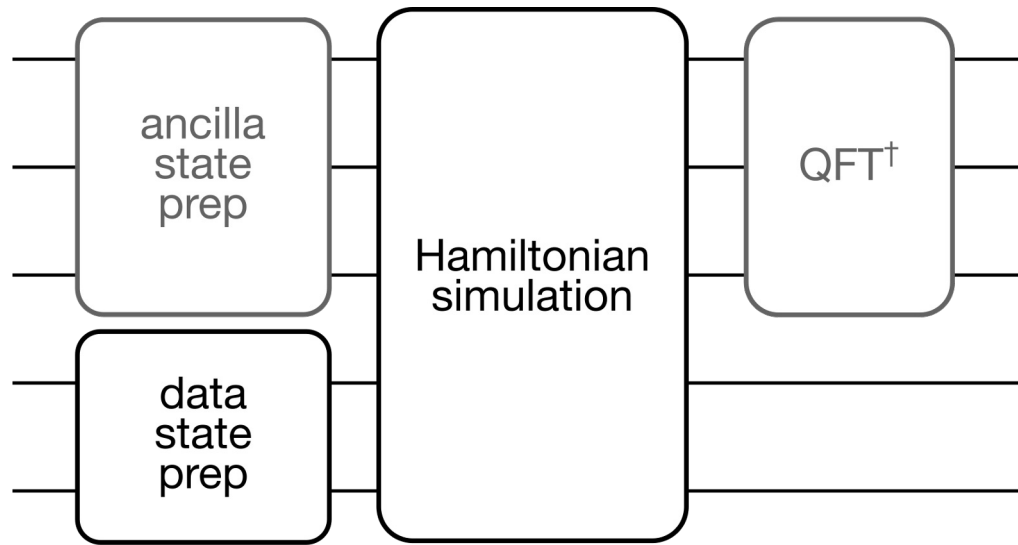
- T: Implemented with magic state preparation from T factories.
- Rotations: Synthesised with different algorithms → more ancilla or T gates.
- Toffoli: AND gate, can be broken down into Clifford and T gates.

T's and Toffolis are challenging: Magic state factories are slow and limited in number.

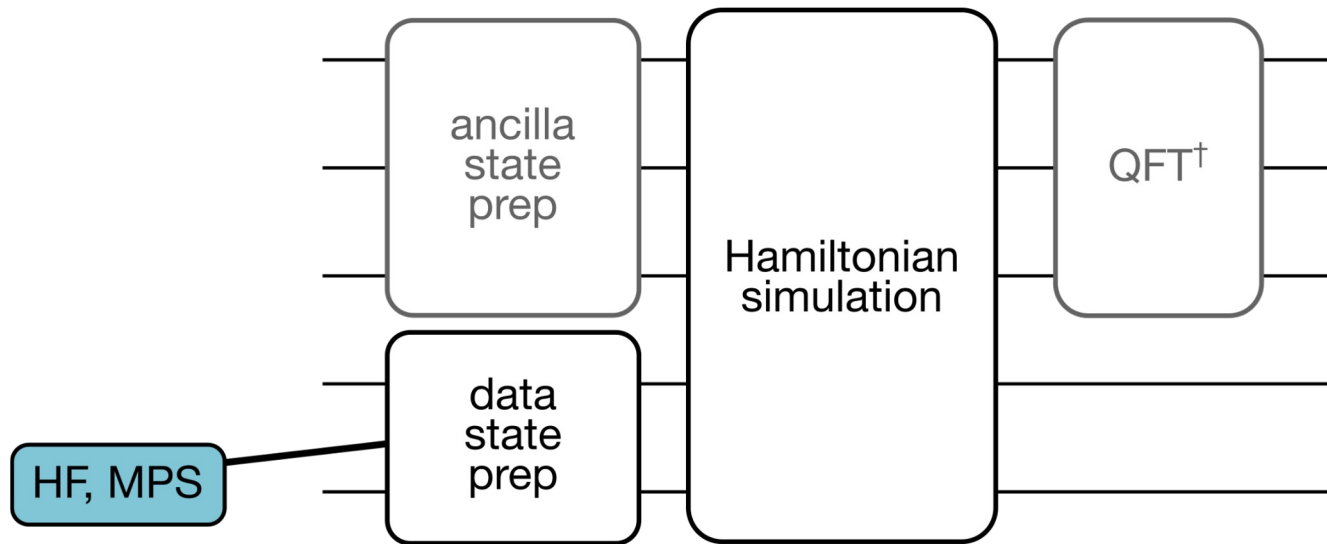
This is often the limiting factor in costing up algorithms.



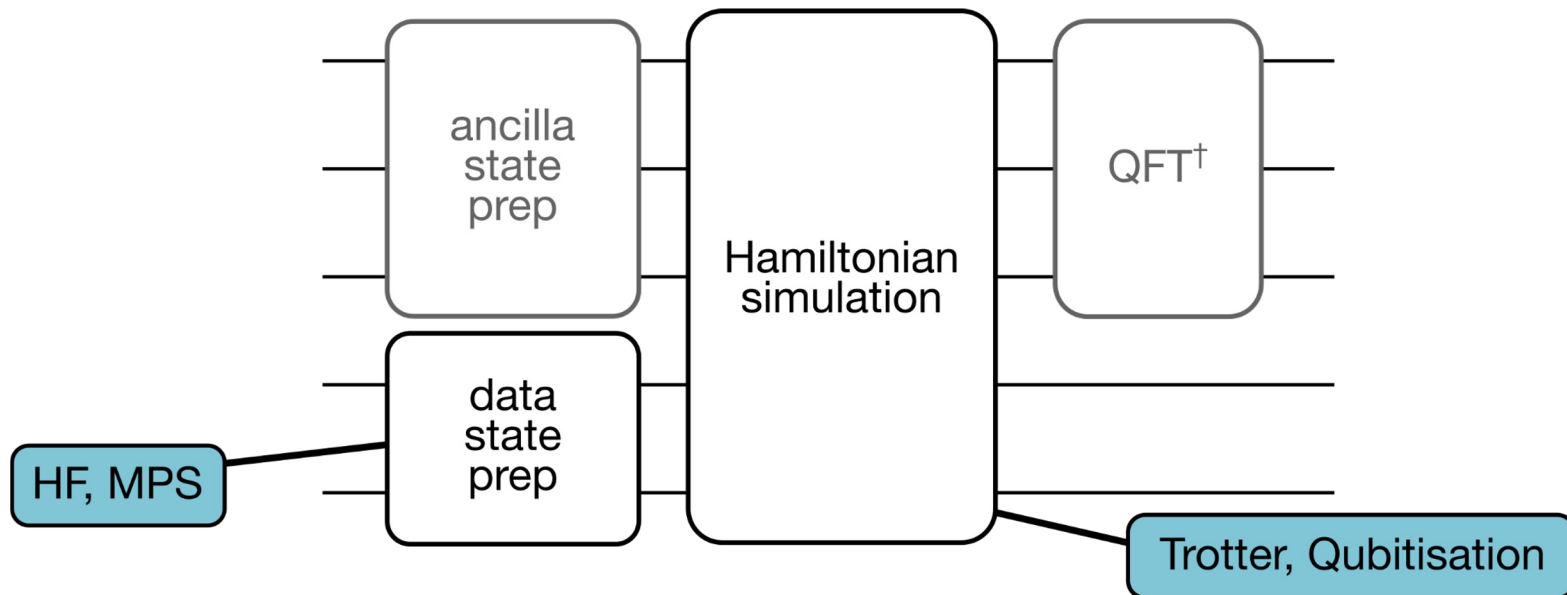
Resource counts for QPE algorithms



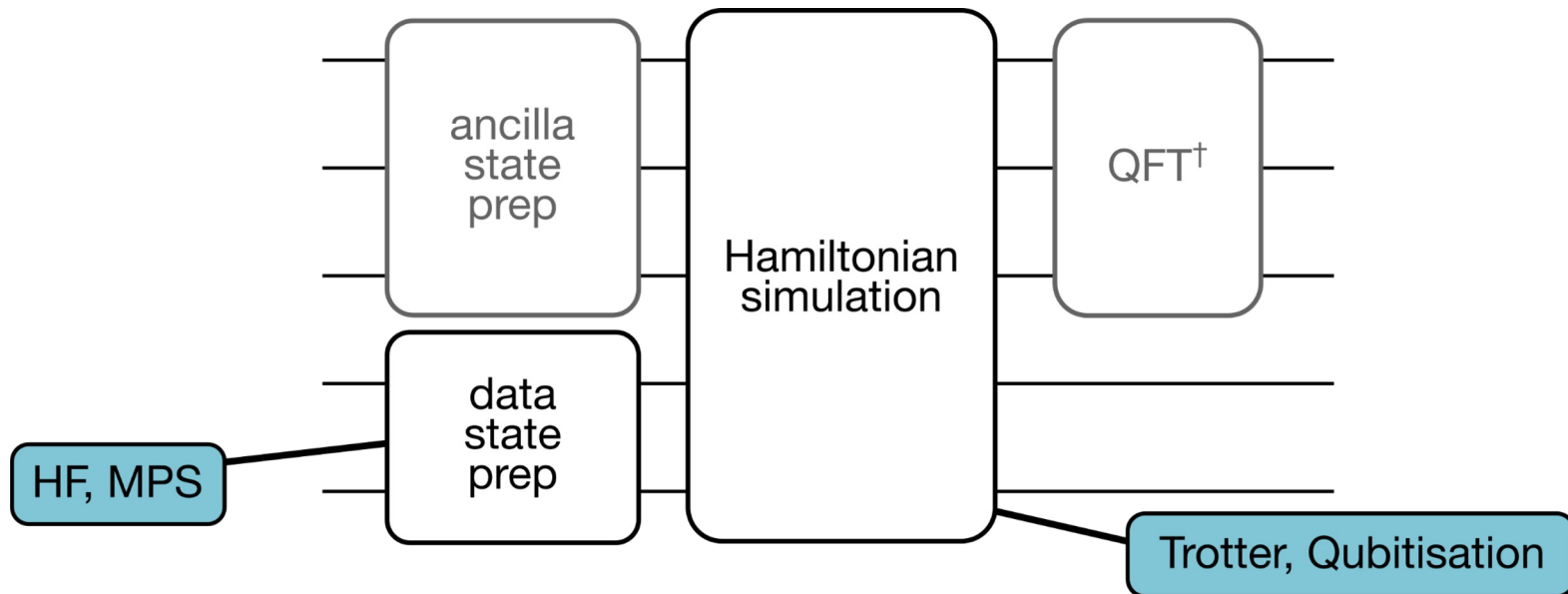
Resource counts for QPE algorithms



Resource counts for QPE algorithms



Resource counts for QPE algorithms



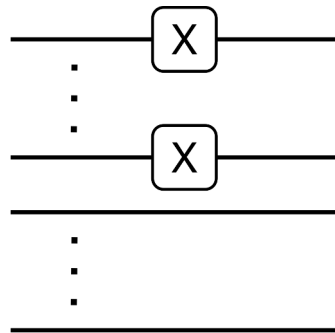
$$\text{Total cost} \sim C(\text{state prep}) + C(\text{Hamiltonian simulation}) + C(\text{QFT}^\dagger)$$

Resources for second quantisation state preparation

Hartree-Fock state

HF state is the occupation number vector with the lowest N_e orbitals occupied.

Resources: $2 N_o$ data qubits, N_e Cliffords.



Beyond-HF states

E.g. states from a DMRG calculation, CASCI states.

Resources: Depend on the specific state used and on the state prep algorithm, but generally will give rise to more ancilla qubits, as well as non-Clifford gates.

Resources for trotterised Hamiltonian simulation

Starting from a Hamiltonian as a sum of Pauli strings, $H = \sum_{j=1}^N c_j P_j$, Trotterisation approximates the time propagator as

$$U(t) = e^{iHt} \approx \prod_{\alpha=1}^L e^{ia_{\alpha} P_{j_{\alpha}}}$$

The cost of a full Trotterisation is the cumulative cost of all Pauli phasors:

$$C(\text{Trotter}) \sim \sum_{\alpha=1}^L C(e^{ia_{\alpha} P_{j_{\alpha}}})$$

Unless stated otherwise, we assume that **gates are applied sequentially**.

Resources for trotterised Hamiltonian simulation

Starting from a Hamiltonian as a sum of Pauli strings, $H = \sum_{j=1}^N c_j P_j$, Trotterisation approximates the time propagator as

$$U(t) = e^{iHt} \approx \prod_{\alpha=1}^L e^{ia_{\alpha} P_{j_{\alpha}}}$$

We will now estimate resources for the Pauli phasors

The cost of a full Trotterisation is the cumulative cost of all Pauli phasors:

$$C(\text{Trotter}) \sim \sum_{\alpha=1}^L C(e^{ia_{\alpha} P_{j_{\alpha}}})$$

Unless stated otherwise, we assume that **gates are applied sequentially**.

Resources for Pauli phasors - make everything Z's

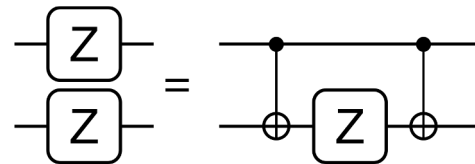
Goal: understand the breakdown of a general phasor $e^{i\theta P}$, where P is some Pauli string, into Clifford and rotation gates.

We use that

$$\begin{aligned} X &= HZH & \text{---} [X] \text{---} &= \text{---} [H] [Z] [H] \text{---} \\ Y &= SHZHS^\dagger & \text{---} [Y] \text{---} &= \text{---} [S^\dagger] [H] [Z] [H] [S] \text{---} \end{aligned}$$

Any Pauli string can therefore be written as a string of all Z's, sandwiched between Hadamard and phase gates.

We can reduce the Z gates to a single one using CNOTs:

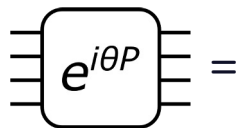


Resources for Pauli phasors - Rotate the Z

Note that for a single Z gate, its exponential $e^{i\theta Z}$ is a rotation gate: $R_Z(\theta)$

With this, we have all the ingredients to build the circuit!

$$P = Z_1 Y_2 X_4$$



Resources for Pauli phasors - Rotate the Z

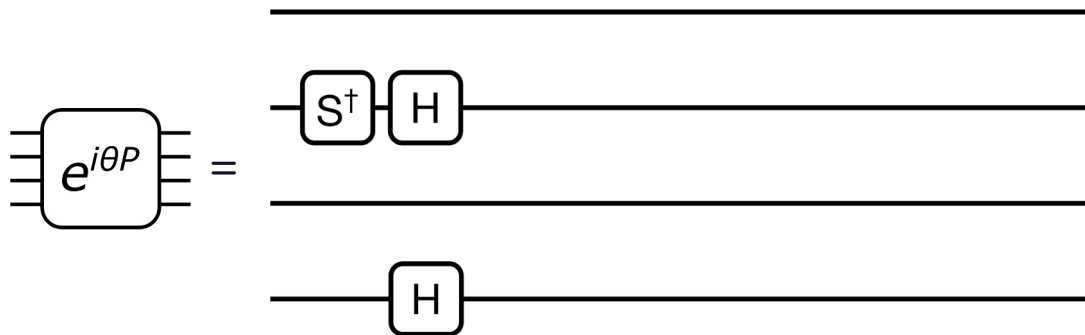
Note that for a single Z gate, its exponential $e^{i\theta Z}$ is a rotation gate: $R_Z(\theta)$

With this, we have all the ingredients to build the circuit!

$$P = Z_1 Y_2 X_4$$

Steps:

1. Transform X and Y into Z



Resources for Pauli phasors - Rotate the Z

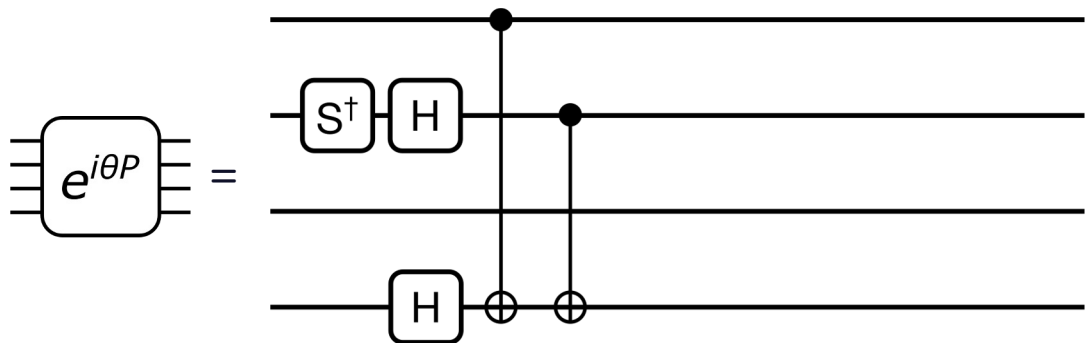
Note that for a single Z gate, its exponential $e^{i\theta Z}$ is a rotation gate: $R_Z(\theta)$

With this, we have all the ingredients to build the circuit!

$$P = Z_1 Y_2 X_4$$

Steps:

1. Transform X and Y into Z
2. Entangle the Z's using CNOT



Resources for Pauli phasors - Rotate the Z

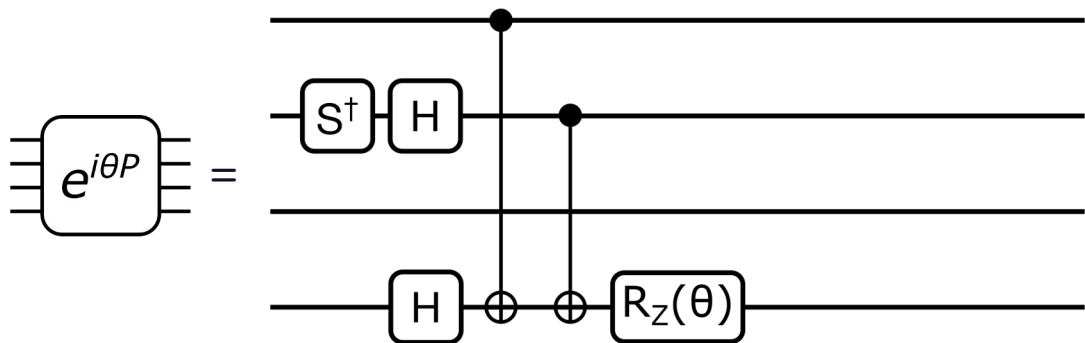
Note that for a single Z gate, its exponential $e^{i\theta Z}$ is a rotation gate: $R_Z(\theta)$

With this, we have all the ingredients to build the circuit!

$$P = Z_1 Y_2 X_4$$

Steps:

1. Transform X and Y into Z
2. Entangle the Z's using CNOT
3. Rotate Z



Resources for Pauli phasors - Rotate the Z

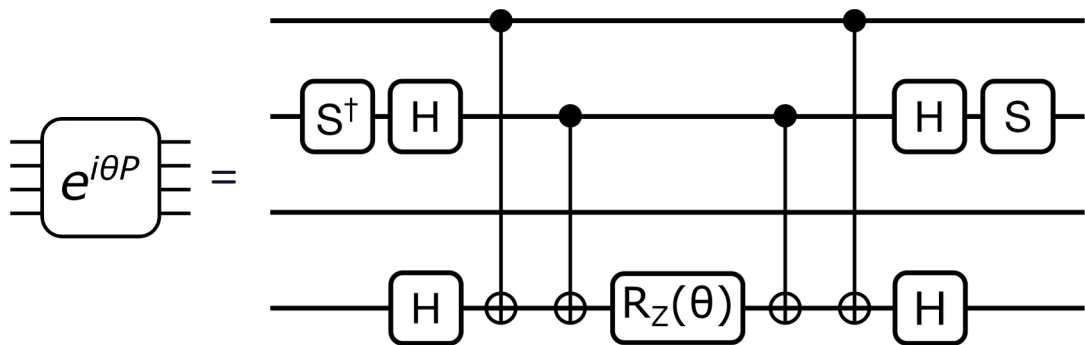
Note that for a single Z gate, its exponential $e^{i\theta Z}$ is a rotation gate: $R_Z(\theta)$

With this, we have all the ingredients to build the circuit!

$$P = Z_1 Y_2 X_4$$

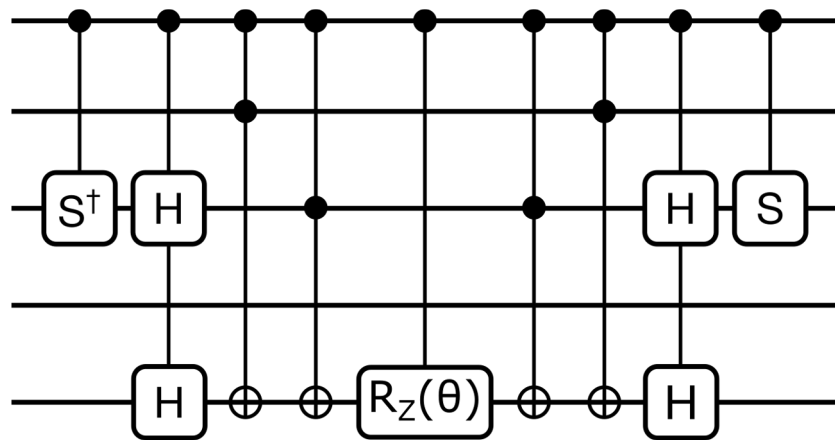
Steps:

1. Transform X and Y into Z
2. Entangle the Z's using CNOT
3. Rotate Z
4. Undo steps 1 and 2



Resources for controlled Pauli phasors

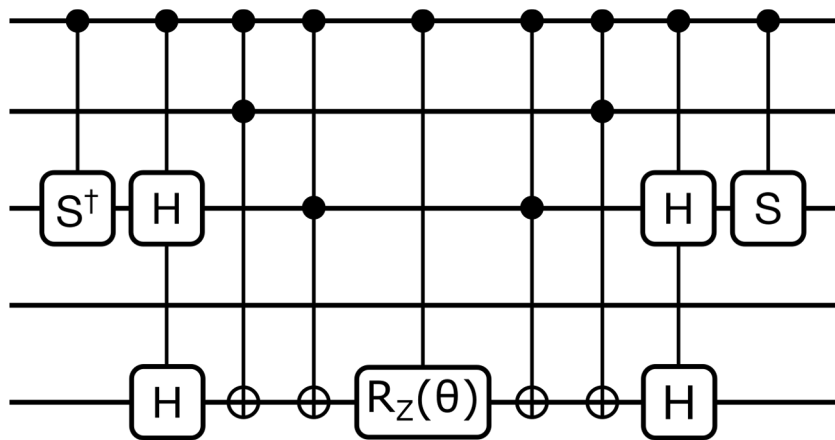
Naively, we could make all operations into controlled ones:



... but this gives rise to many hard gates.

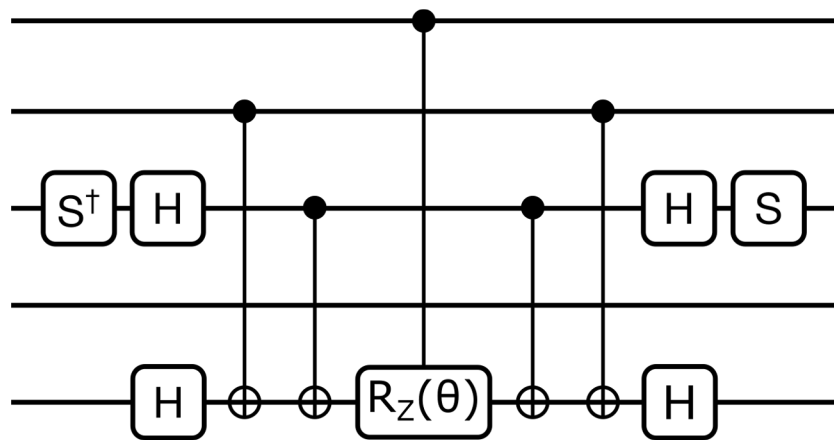
Resources for controlled Pauli phasors

Naively, we could make all operations into controlled ones:



... but this gives rise to many hard gates.

Instead, we only need to control the rotation:

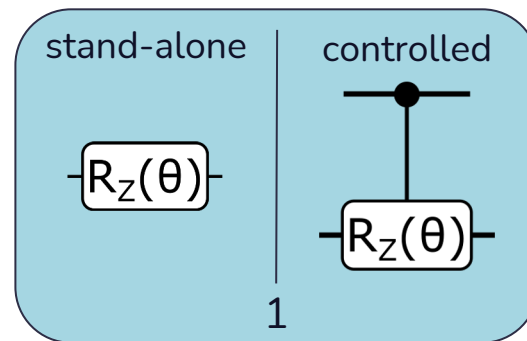
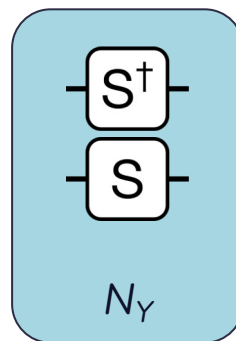
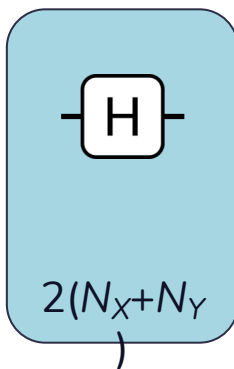
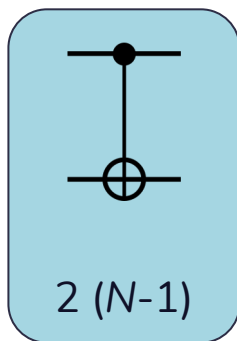


Final bookkeeping for Pauli phasors

We can use our insight to find a general accounting scheme. First, analyse the Pauli string to find

- Its Pauli weight $\rightarrow N$
- The numbers of X and Y Paulis $\rightarrow N_X, N_Y$

Resources:

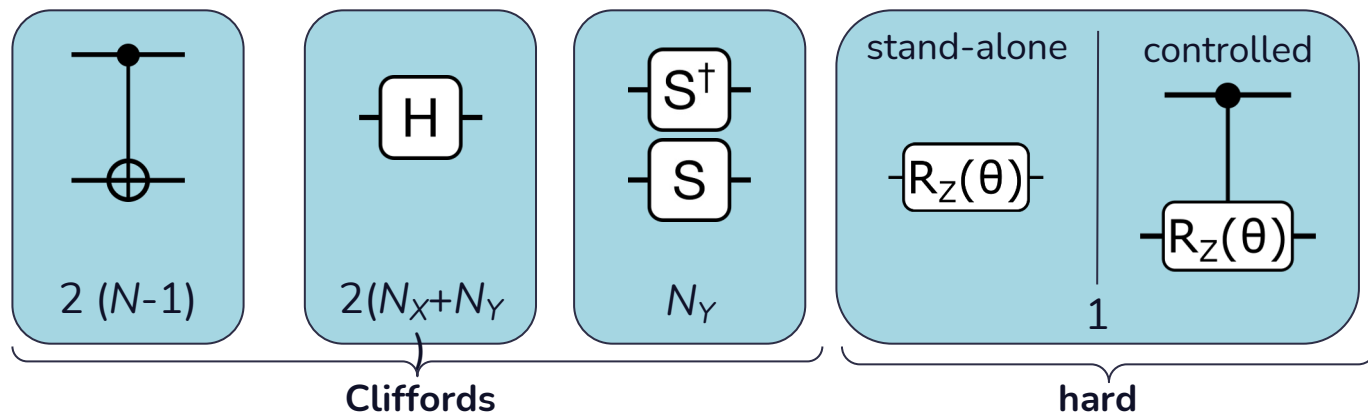


Final bookkeeping for Pauli phasors

We can use our insight to find a general accounting scheme. First, analyse the Pauli string to find

- Its Pauli weight $\rightarrow N$
- The numbers of X and Y Paulis $\rightarrow N_X, N_Y$

Resources:



Resources for a Trotterisation

To account for the resources of a Trotterisation, loop over all L terms and cost each Pauli phasor separately

$$U(t) = e^{iHt} \approx \prod_{\alpha=1}^L e^{ia_{\alpha} P_{j_{\alpha}}} \quad C(\text{Trotter}) \sim \sum_{\alpha=1}^L C(e^{ia_{\alpha} P_{j_{\alpha}}})$$

- Upper limit can be found by assuming all Paulis to be Y , but might overestimate the resources significantly.
- Trotterisation uses many Cliffords, but few non-Clifford gates.
- Well-suited when T gates should be avoided.
- Many of the rotation angles are the same, which can further reduce resources.

Qubitised Hamiltonian simulation

Recall: Given an LCU block encoding of the Hamiltonian $U_H = \text{PREP}^\dagger \cdot \text{SELECT} \cdot \text{PREP}$, qubitisation uses the walk operator

$$W = \text{PREP}^\dagger \cdot \text{SELECT} \cdot \text{PREP} (2|0_{\text{anc}}\rangle\langle 0_{\text{anc}}| - 1)$$

for Hamiltonian simulation.

Qubitised Hamiltonian simulation

Recall: Given an LCU block encoding of the Hamiltonian $U_H = \text{PREP}^\dagger \cdot \text{SELECT} \cdot \text{PREP}$, qubitisation uses the walk operator

$$W = \text{PREP}^\dagger \cdot \text{SELECT} \cdot \text{PREP} (2|0_{\text{anc}}\rangle\langle 0_{\text{anc}}| - 1)$$

**multi-CZ on the ancilla:
Focus only on block
encoding below**

for Hamiltonian simulation.

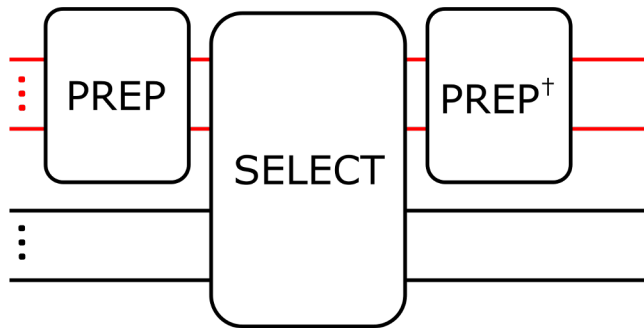
Qubitised Hamiltonian simulation

Recall: Given an LCU block encoding of the Hamiltonian $U_H = \text{PREP}^\dagger \cdot \text{SELECT} \cdot \text{PREP}$, qubitisation uses the walk operator

$$W = \text{PREP}^\dagger \cdot \text{SELECT} \cdot \text{PREP} (2|0_{\text{anc}}\rangle\langle 0_{\text{anc}}| - 1)$$

**multi-CZ on the ancilla:
Focus only on block
encoding below**

for Hamiltonian simulation.



$$C(\text{LCU}) \sim C(\text{PREP}) + C(\text{PREP}^\dagger) + C(\text{SELECT})$$

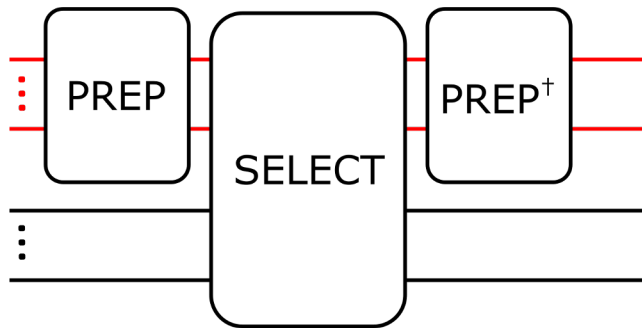
Qubitised Hamiltonian simulation

Recall: Given an LCU block encoding of the Hamiltonian $U_H = \text{PREP}^\dagger \cdot \text{SELECT} \cdot \text{PREP}$, qubitisation uses the walk operator

$$W = \text{PREP}^\dagger \cdot \text{SELECT} \cdot \text{PREP} (2|0_{\text{anc}}\rangle\langle 0_{\text{anc}}| - 1)$$

**multi-CZ on the ancilla:
Focus only on block
encoding below**

for Hamiltonian simulation.



$$C(\text{LCU}) \sim C(\text{PREP}) + C(\text{PREP}^\dagger) + C(\text{SELECT})$$

Next: Find $C(\text{PREP})$ and $C(\text{SELECT})$.

Resources for the PREP operation

PREP can be any state preparation algorithm, $\text{PREP}|0_{\text{anc}}\rangle = \sum_{j=1}^N \sqrt{\frac{c_j}{c}} |j\rangle$

For example, using rotations:

$$N = 2 \quad \text{PREP}|0_{\text{anc}}\rangle = \cos \theta |0\rangle + \sin \theta |1\rangle \quad \boxed{\text{R}_Y(\theta)}$$

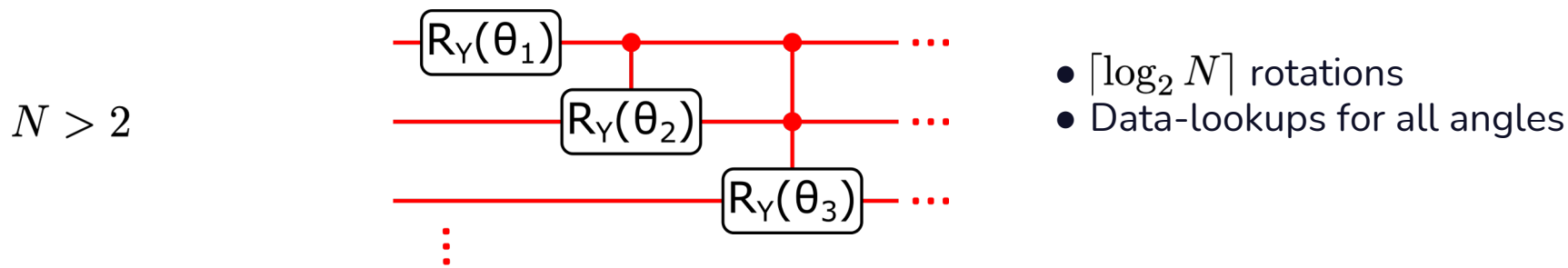
- 1 rotation
- Data-lookup for θ

Resources for the PREP operation

PREP can be any state preparation algorithm, $\text{PREP}|0_{\text{anc}}\rangle = \sum_{j=1}^N \sqrt{\frac{c_j}{c}} |j\rangle$

For example, using rotations:

$$N = 2 \quad \text{PREP}|0_{\text{anc}}\rangle = \cos \theta |0\rangle + \sin \theta |1\rangle \quad \boxed{R_Y(\theta)} \quad \begin{array}{l} \bullet 1 \text{ rotation} \\ \bullet \text{Data-lookup for } \theta \end{array}$$

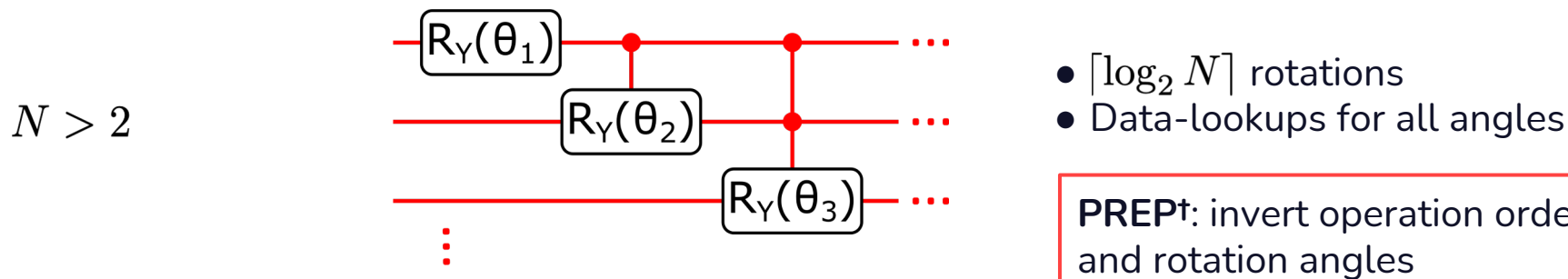


Resources for the PREP operation

PREP can be any state preparation algorithm, $\text{PREP}|0_{\text{anc}}\rangle = \sum_{j=1}^N \sqrt{\frac{c_j}{c}} |j\rangle$

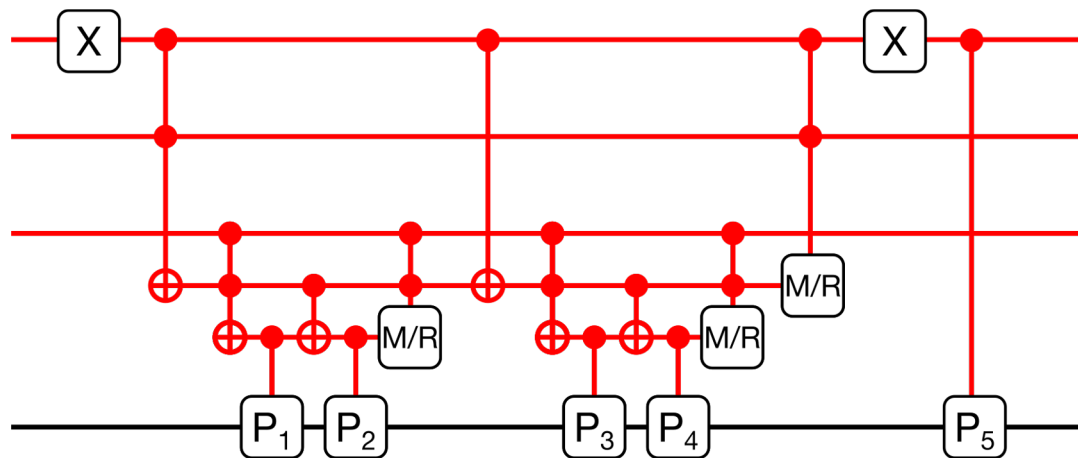
For example, using rotations:

$$N = 2 \quad \text{PREP}|0_{\text{anc}}\rangle = \cos \theta |0\rangle + \sin \theta |1\rangle \quad \boxed{R_Y(\theta)} \quad \begin{array}{l} \bullet \text{ 1 rotation} \\ \bullet \text{ Data-lookup for } \theta \end{array}$$

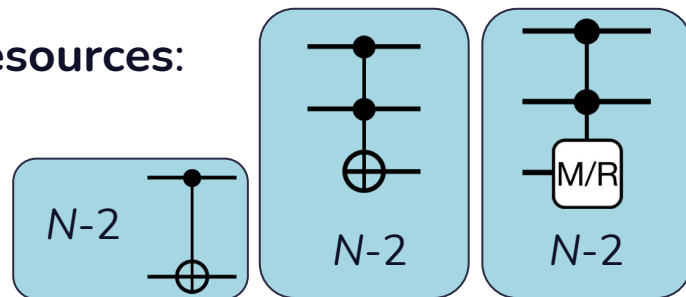


Resources for the SELECT operation

The SELECT operator is an example of a unary iterator, which steps through the list $j = 1, \dots, N$ and selects a unitary from a list to apply for index j .

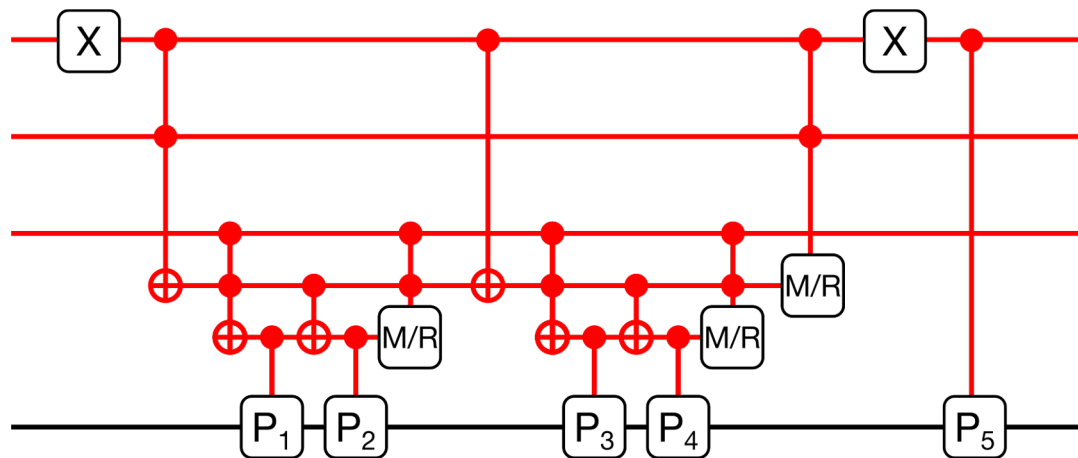


Resources:

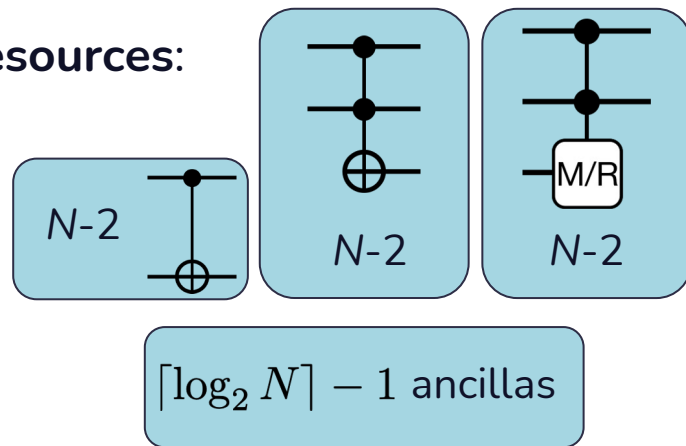


Resources for the SELECT operation

The SELECT operator is an example of a unary iterator, which steps through the list $j = 1, \dots, N$ and selects a unitary from a list to apply for index j .

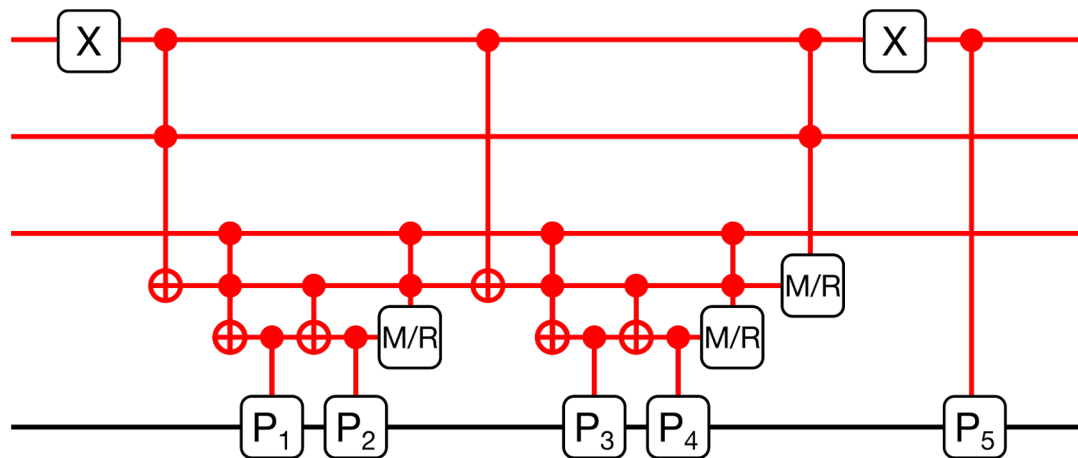


Resources:

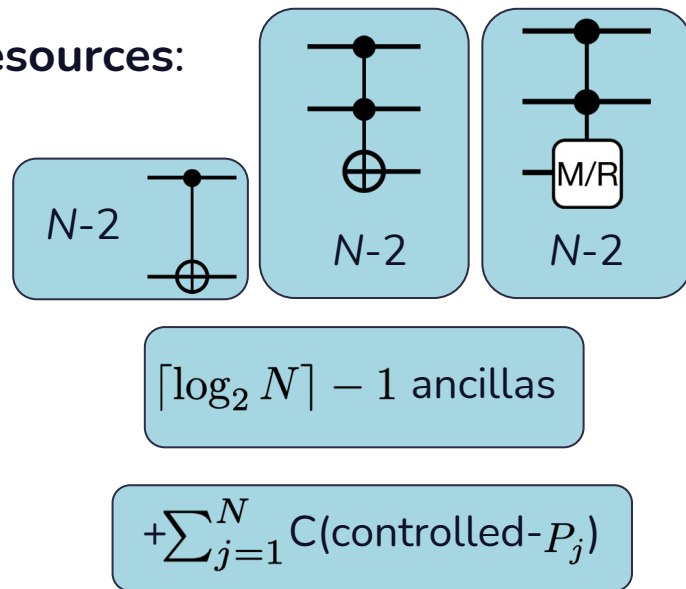


Resources for the SELECT operation

The SELECT operator is an example of a unary iterator, which steps through the list $j = 1, \dots, N$ and selects a unitary from a list to apply for index j .

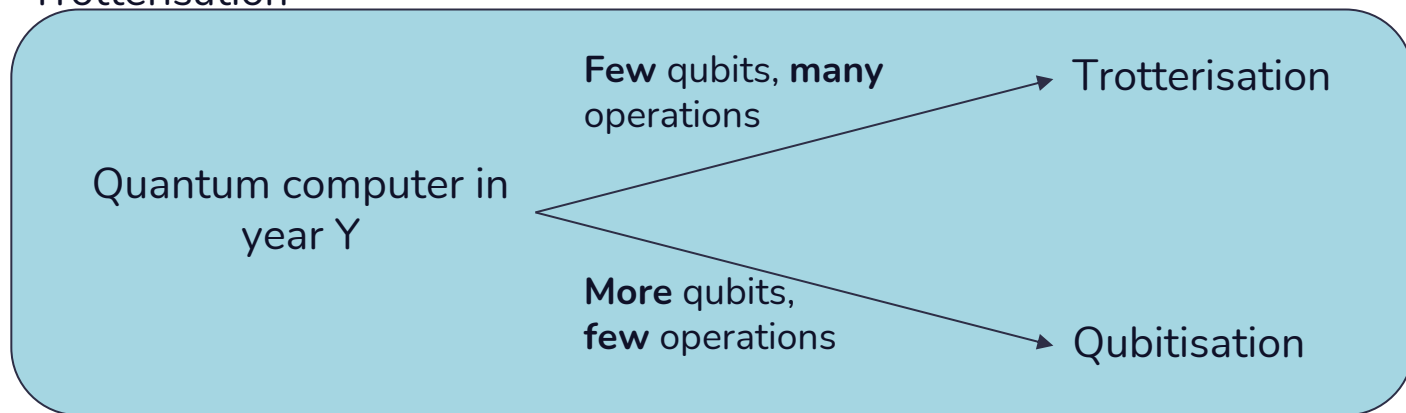


Resources:



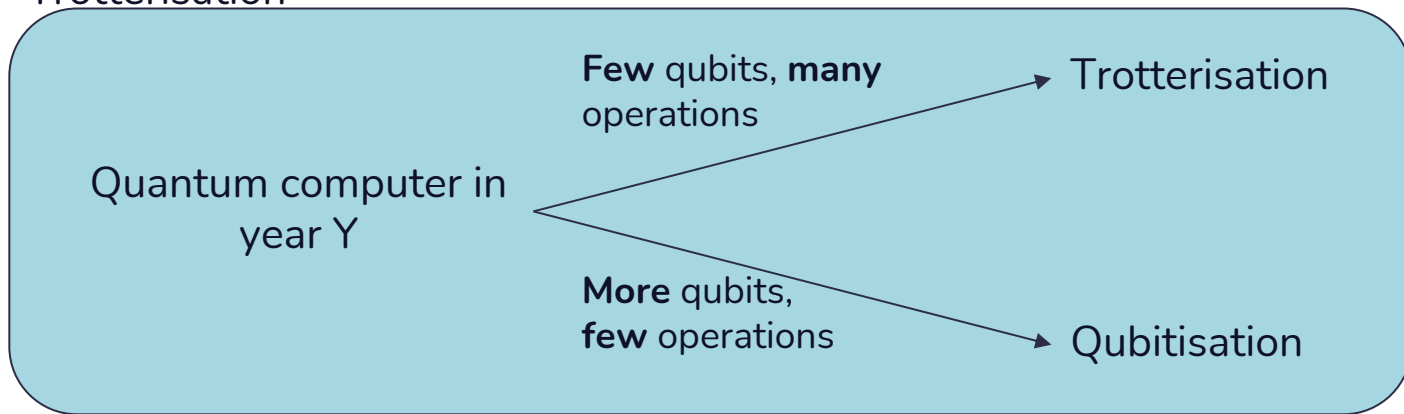
Total qubitisation cost and Trotterisation comparison

- Specific cost depends on the state preparation algorithm and the Pauli strings
- Generally speaking, qubitisation produces shallower but wider circuits than Trotterisation



Total qubitisation cost and Trotterisation comparison

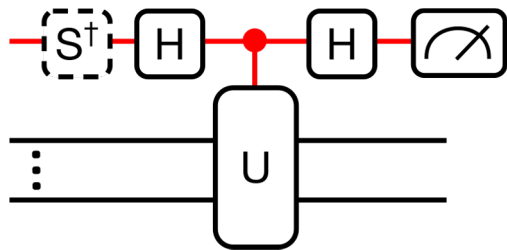
- Specific cost depends on the state preparation algorithm and the Pauli strings
- Generally speaking, qubitisation produces shallower but wider circuits than Trotterisation



... **algorithmic improvements** continue shifting the balance!

Resource counts for QPE algorithms

Hadamard test

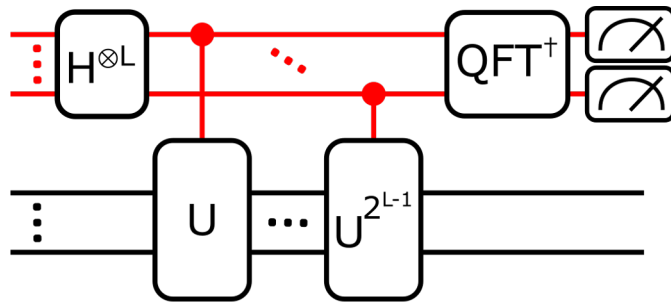


Permanent qubits: 1 ancilla, N data

Gates:

- State preparation
 - 1 C-Ham. simulation
 - 2-3 Cliffords
- } **many repetitions**

Textbook QPE



Permanent qubits: L ancilla, N data

Gates:

- State preparation
 - $2^L - 1$ C-Ham. simulation
 - Inverse QFT
- } **few repetitions**

Renormalising the Hamiltonian

From the morning session: QPE only determines phases modulo 1. To avoid looping around and losing information, renormalise the Hamiltonian such that the important eigenvalues are between 0 and 1.

Renormalised propagator: $e^{iHt/\lambda}$

One effect on QPE is an increase in ancilla qubits, given a target accuracy: $\varepsilon_{\text{target}} = 1.6 \text{ mHa}$

The QPE energy is multiplied by λ , therefore $\varepsilon_{\text{QPE}} = \varepsilon_{\text{target}}/\lambda$

Goal: Keep λ as small as possible!

Quantum phase estimation - setup

Eigenvalues of H become phases of a unitary

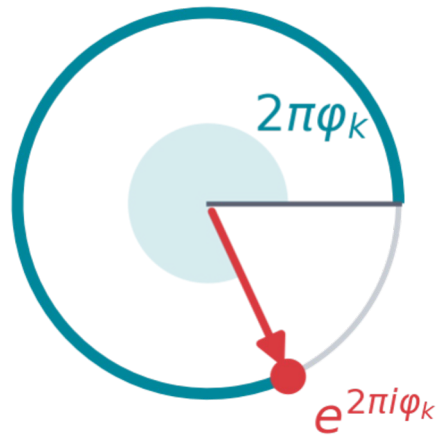
$$U = e^{i\hat{H}t} \implies U |\phi_k\rangle = e^{iE_k t} |\phi_k\rangle$$

First, some rearrangement: we focus on a phase φ_k between zero and one

$$e^{iE_k t} = e^{2\pi i \varphi_k}, \quad \varphi_k = \frac{E_k t}{2\pi} \bmod 1 \quad E_k = \frac{2\pi}{t} \varphi_k \bmod \frac{2\pi}{t}$$

Using QPE, we can obtain φ_k . But what value should we set for t ? Too large and φ_k will “loop around” as it is modulo 1, too small and our precision suffers.

Solution: renormalise H so that the eigenphases we care about are between zero and one - requires some approximations to the eigenspectrum of H .



Advanced topics

Shots

A measurement from a quantum computer is always stochastic. To be sure that the correct energy is estimated from QPE, the circuit may need to be repeated.

$$\text{Total cost} \sim \text{Number of shots} \cdot \text{Cost of single repetition}$$

To physical resources

Qubits: Error correction code assigns a number of physical qubits to a single logical qubit (e.g. surface code: $2d^2 - 1$ physical qubits per logical qubit). Additional qubits are needed e.g. for magic state factories.

Gates: Decompose gates into physical instructions, very hardware-specific.

Summary and outlook

Takeaways:

- Importance of resource estimation for algorithm development and understanding quantum computing roadmaps.
- Hands-on demonstration of resource estimation workflow, demonstrated on building blocks of the QPE workflow.

Next up:

- Introduction to QUICHE, a tool for resource estimation for chemistry interfaced with ORCA.
- Make resource estimation accessible for chemists interested in quantum algorithms.
- Including a live demo!!